

Treinamento de Inteligência Artificial

Google Colab + Kaggle

Projeto1.

Autor : Luis Fernando Almeida De Oliveira

Treinamento usando Python

Primeiro Projeto usando Machine Learning.

1. Introdução

Nos últimos anos, a Inteligência Artificial tornou-se uma ferramenta cada vez mais importante para a análise de imagens, a automatização de processos e a tomada de decisões em diferentes setores produtivos. Dentro desse campo, a visão computacional permite que os modelos de IA analisem imagens digitais e aprendam a identificar padrões visuais, como formas, cores, bordas e estruturas.

Um dos problemas em que esse tipo de tecnologia pode ser aplicado é a detecção de logos reais e falsos. Em áreas como o design gráfico, o marketing digital, o comércio eletrônico e a proteção da identidade visual das marcas, a identificação de logos alterados ou gerados pode ser útil como uma ferramenta de apoio para reconhecer possíveis irregularidades em conteúdos visuais.

O presente trabalho tem como objetivo treinar um modelo de Inteligência Artificial capaz de classificar imagens de logos em duas categorias: reais e falsos. Para isso, foi utilizado o dataset “Fake/Real Logo Detection Dataset”, disponível no Kaggle, e o treinamento foi realizado no Google Colab utilizando Python e a biblioteca TensorFlow/Keras.

É importante destacar que o modelo desenvolvido não tem como finalidade atuar como um verificador oficial de autenticidade de marcas. Sua função é realizar uma classificação binária baseada nos padrões visuais aprendidos a partir do conjunto de dados utilizado. Portanto, o projeto busca não apenas treinar um modelo de IA, mas também analisar seus resultados, interpretar suas métricas de desempenho, reconhecer suas limitações e propor possíveis melhorias futuras.

Dessa forma, o trabalho permite compreender de maneira prática como se desenvolve um processo básico de treinamento de um modelo de Inteligência Artificial aplicado a imagens, desde a seleção do dataset até a avaliação dos resultados obtidos.

2. Objetivo do modelo

O objetivo do modelo desenvolvido neste projeto é realizar uma classificação binária de imagens de logos, diferenciando entre duas categorias principais: **logos reais** e **logos falsos**. Para isso, foi utilizado um conjunto de dados previamente rotulado, no qual as imagens foram organizadas em duas classes: real e fake.

O modelo foi treinado para analisar características visuais presentes nas imagens, como formas, cores, bordas e estruturas gerais dos logos. A partir desses padrões, a Inteligência Artificial tenta prever se uma imagem pertence à classe de logos reais ou à classe de logos falsos.

É importante destacar que o modelo não tem como objetivo identificar a marca do logo. Ou seja, ele não foi treinado para responder se uma imagem corresponde à Nike, Adidas, Apple, Google ou a outra empresa. Sua função limita-se a classificar a imagem entre as categorias **real** ou **fake**.

Além disso, o modelo também não deve ser interpretado como um verificador oficial de autenticidade de marcas. A classificação realizada baseia-se apenas nos padrões aprendidos durante o treinamento com o dataset utilizado. Portanto, uma imagem classificada como “real” pelo modelo não necessariamente representa um logo oficialmente autêntico do ponto de vista legal ou comercial.

Dessa forma, o objetivo principal do modelo é demonstrar como uma rede neural convolucional pode ser aplicada a um problema de visão computacional, utilizando imagens de logos para treinar, validar e avaliar um sistema básico de classificação automática.

3. Descrição do dataset

Para o desenvolvimento deste projeto, foi utilizado o dataset **Fake/Real Logo Detection Dataset**, disponível na plataforma Kaggle. Esse conjunto de dados contém imagens de logos organizadas com o objetivo de trabalhar em uma tarefa de classificação entre logos reais e logos falsos ou gerados.

Ao analisar a estrutura original do dataset, foram identificadas duas pastas principais: `genLogoOutput` e `output`. Para o desenvolvimento do modelo, a pasta `genLogoOutput` foi interpretada como a classe correspondente a logos falsos ou gerados, enquanto a pasta `output` foi interpretada como a classe correspondente a logos reais.

Com o objetivo de facilitar o treinamento do modelo no TensorFlow/Keras, as imagens foram reorganizadas em uma nova estrutura de pastas chamada `dataset_final`, dividida em duas classes principais:

```
dataset_final/  
  fake/  
  real/
```

Após a reorganização, o conjunto final ficou composto por um total de 825 imagens, distribuídas da seguinte forma:

```
fake: 550 imagens  
real: 275 imagens
```

Essa distribuição mostra que o dataset está desbalanceado, pois contém uma quantidade maior de imagens classificadas como falsas do que imagens classificadas como reais. Esse aspecto é importante porque pode influenciar o comportamento do modelo durante o treinamento e os resultados obtidos durante a validação.

O dataset inclui logos de diferentes marcas e setores, como tecnologia, alimentos, redes sociais, automóveis, bebidas e produtos de consumo. Essa variedade permite que o modelo trabalhe com diferentes formas, cores e estruturas visuais, o que é útil para uma primeira aproximação ao problema de classificação de logos.

A relevância desse conjunto de dados está relacionada à sua possível aplicação em setores como design gráfico, marketing digital, comércio eletrônico e proteção da identidade visual das marcas. Embora o modelo desenvolvido não funcione como um verificador oficial de autenticidade, o dataset permite explorar como uma Inteligência Artificial pode aprender padrões visuais e utilizá-los para classificar imagens em categorias previamente definidas.

4. Análise exploratória de dados (EDA)

Antes de realizar o treinamento do modelo, foi feita uma análise exploratória do conjunto de dados com o objetivo de compreender sua estrutura, a quantidade de imagens disponíveis e a distribuição das classes.

Em primeiro lugar, foi revisada a organização original do dataset baixado do Kaggle. Foram identificadas duas pastas principais: genLogoOutput e output. A pasta genLogoOutput continha imagens de logos gerados ou falsos, enquanto a pasta output continha imagens correspondentes a logos reais.

Posteriormente, foi realizada uma contagem das imagens disponíveis em cada grupo. O resultado obtido foi o seguinte:

genLogoOutput: 550 imagens

output: 275 imagens

A partir dessas informações, as imagens foram reorganizadas em duas classes mais claras para o treinamento do modelo:

fake: 550 imagens

real: 275 imagens

O total do conjunto de dados utilizado foi de **825 imagens**.

Essa análise permitiu identificar que o dataset estava desbalanceado, já que a classe fake possuía o dobro de imagens em comparação com a classe real. Esse ponto é relevante porque um conjunto de dados desbalanceado pode influenciar o aprendizado do modelo, fazendo com que ele tenha maior facilidade para reconhecer a classe com maior quantidade de exemplos.

Durante a análise, também foi observado que o dataset incluía logos de diferentes marcas e setores, como tecnologia, alimentos, automóveis, redes sociais, bebidas e produtos de consumo. Essa variedade visual permite que o modelo trabalhe com diferentes formas, cores, estruturas e padrões gráficos.

Além disso, as imagens foram redimensionadas para **70x70 pixels** para facilitar seu processamento durante o treinamento. Essa redução permite diminuir o custo computacional, embora também possa provocar perda de detalhes visuais importantes. Por esse motivo, a resolução das imagens foi considerada uma possível limitação do modelo.

Em conclusão, a análise exploratória permitiu compreender melhor a composição do dataset, identificar o desbalanceamento entre as classes e reconhecer alguns fatores que poderiam influenciar o desempenho final do modelo.

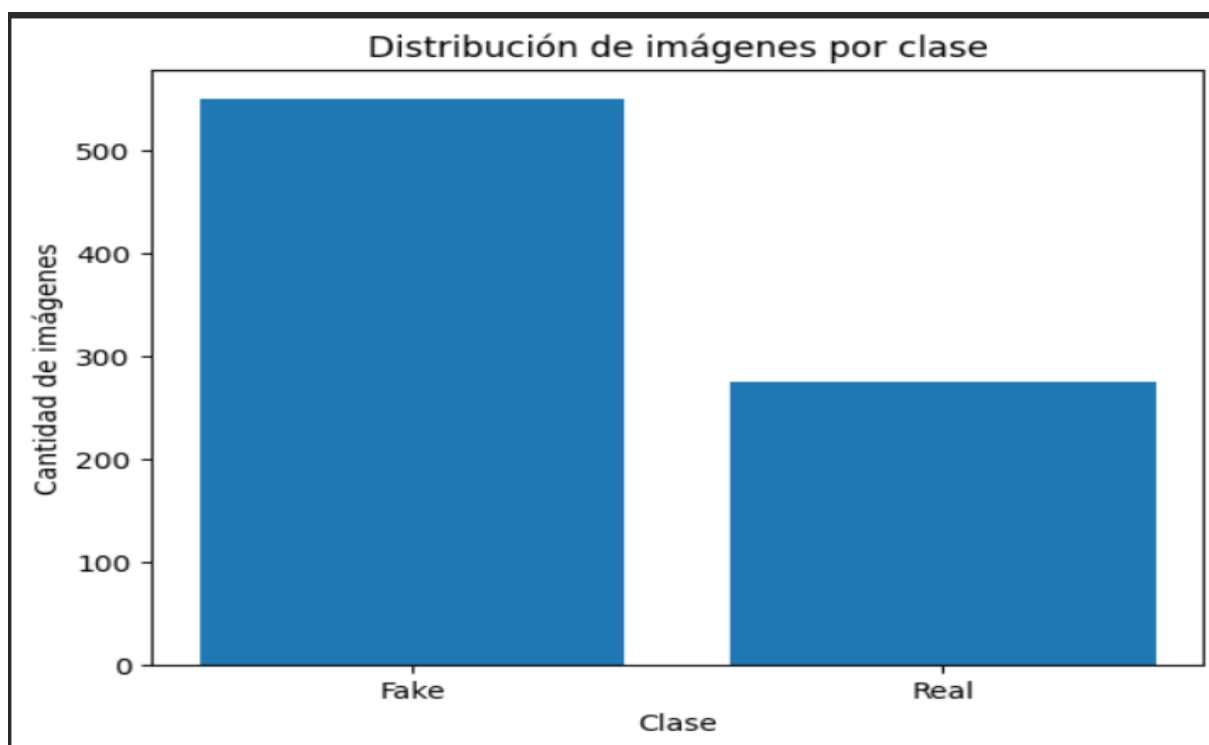


Imagem1. Distribuição das imagens nas classes do dataSet utilizado.

5. Preparação e pré-processamento dos dados

Após a realização da análise exploratória do dataset, foi feita a preparação dos dados para que pudessem ser utilizados corretamente durante o treinamento do modelo de Inteligência Artificial.

Em primeiro lugar, o dataset foi baixado do Kaggle utilizando o Google Colab. Em seguida, o arquivo compactado foi descompactado dentro do ambiente de trabalho, permitindo o acesso às pastas originais do conjunto de dados.

A estrutura inicial do dataset continha duas pastas principais: genLogoOutput e output. Para facilitar o treinamento do modelo, essas pastas foram reorganizadas em uma nova estrutura chamada dataset_final, separando as imagens em duas classes:

```
dataset_final/  
  fake/  
  real/
```

A pasta fake recebeu as imagens provenientes de genLogoOutput, enquanto a pasta real recebeu as imagens provenientes de output. Essa reorganização foi necessária para que o TensorFlow/Keras pudesse carregar as imagens automaticamente, identificando cada pasta como uma classe diferente.

Depois da organização dos dados, as imagens foram carregadas utilizando a função `image_dataset_from_directory`, que permite ler imagens diretamente a partir de pastas organizadas por classe. Durante esse processo, as imagens foram redimensionadas para **70x70 pixels**, com o objetivo de reduzir o custo computacional e facilitar o treinamento no Google Colab.

Além disso, o dataset foi dividido automaticamente em duas partes:

```
80% para treinamento  
20% para validação
```

O conjunto de treinamento foi utilizado para que o modelo aprendesse os padrões visuais presentes nas imagens, enquanto o conjunto de validação foi utilizado para avaliar o

desempenho do modelo com imagens que não foram usadas diretamente durante o treinamento.

Também foi aplicada uma normalização dos valores dos pixels por meio da camada Rescaling(1./255). Esse processo transforma os valores dos pixels, que originalmente variam de 0 a 255, para um intervalo entre 0 e 1. Essa normalização ajuda o modelo a processar os dados de maneira mais eficiente durante o treinamento.

Em resumo, a preparação dos dados incluiu o download do dataset, a revisão de sua estrutura, a reorganização das pastas, o redimensionamento das imagens, a divisão entre treinamento e validação e a normalização dos valores dos pixels.

6. Modelo de IA utilizado

Para este projeto, foi utilizado um modelo de rede neural convolucional, também conhecido como CNN. Esse tipo de modelo é amplamente utilizado em tarefas de visão computacional, pois permite analisar imagens e aprender padrões visuais, como bordas, formas, cores, texturas e estruturas.

A escolha de uma CNN se justifica porque o problema trabalhado neste projeto está relacionado à classificação de imagens. Como o objetivo era diferenciar logos reais e falsos, foi necessário utilizar um modelo capaz de processar informações visuais e extrair características relevantes das imagens.

O modelo foi construído utilizando a biblioteca **TensorFlow/Keras** no Google Colab. A arquitetura utilizada incluiu uma camada de normalização, camadas convolucionais, camadas de redução espacial, uma camada de achatamento, camadas densas e uma camada final de saída binária.

A primeira etapa do modelo corresponde à normalização dos valores dos pixels por meio da camada Rescaling(1./255). Essa camada converte os valores originais dos pixels, que variam de 0 a 255, para um intervalo entre 0 e 1. Isso facilita o aprendizado do modelo e melhora o processamento dos dados durante o treinamento.

Posteriormente, foram utilizadas várias camadas Conv2D, responsáveis por extrair características visuais das imagens. Essas camadas permitem que o modelo identifique padrões como linhas, formas, bordas e combinações visuais mais complexas. Após cada camada convolucional, foi aplicada uma camada MaxPooling2D, cuja função é reduzir o tamanho da informação processada, preservando as características mais importantes.

Em seguida, foi utilizada uma camada Flatten, que transforma as informações extraídas pelas camadas convolucionais em um formato adequado para as camadas densas. Depois disso, foi adicionada uma camada Dense com ativação relu, permitindo que o modelo aprendesse relações mais complexas entre as características extraídas.

Também foi utilizada uma camada Dropout, com o objetivo de reduzir o risco de sobreajuste. O sobreajuste ocorre quando o modelo aprende muito bem os dados de treinamento, mas apresenta dificuldades para generalizar com imagens novas.

Por fim, o modelo utilizou uma camada de saída Dense com uma única neurônio e ativação sigmoid. Essa configuração é adequada para problemas de classificação binária, já que o modelo deve escolher entre duas classes possíveis: real ou fake.

Em resumo, foi escolhida uma rede neural convolucional porque o projeto trabalha com imagens e esse tipo de arquitetura é apropriado para tarefas de classificação visual. O modelo permite analisar padrões presentes nos logos e classificá-los conforme as duas categorias definidas no conjunto de dados.

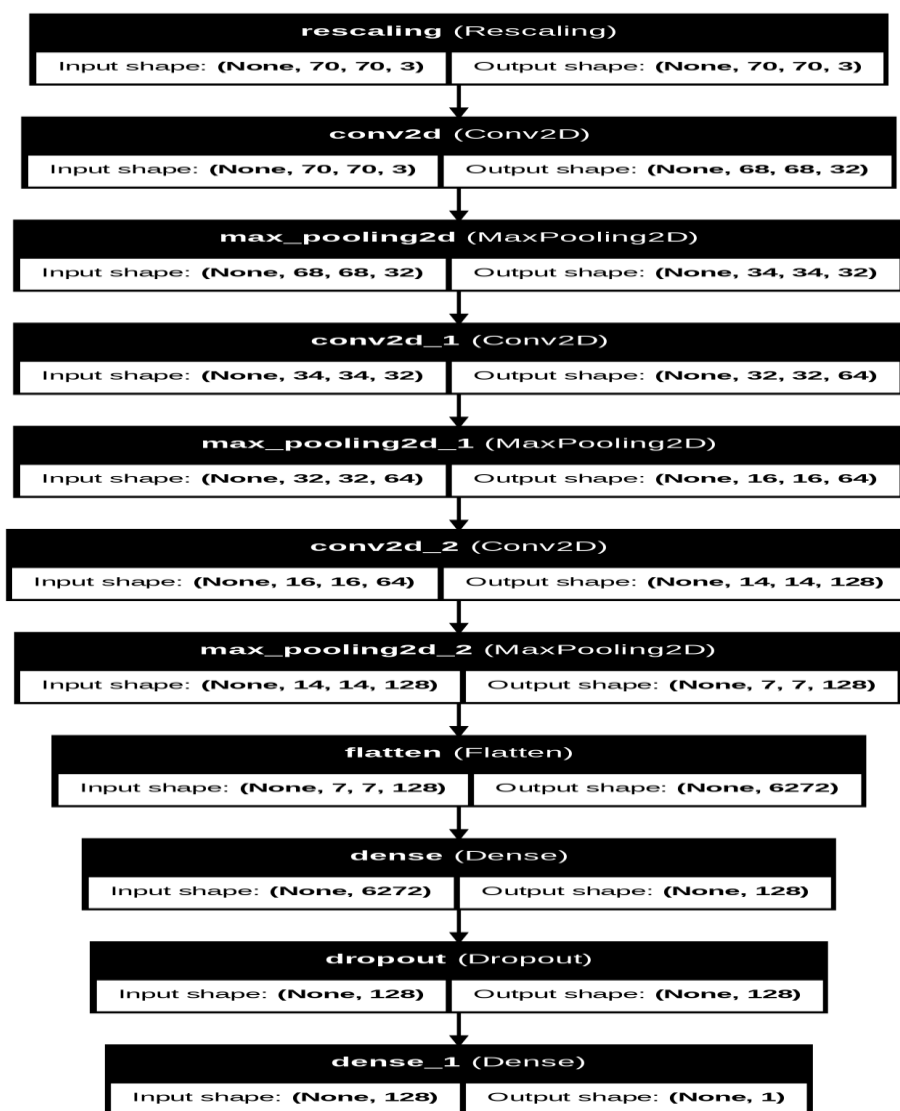


Imagem2. Arquitetura do Projeto.

7. Treinamento do modelo

Após preparar o conjunto de dados e definir a arquitetura da rede neural convolucional, foi realizado o treinamento do modelo no Google Colab utilizando Python e TensorFlow/Keras.

O dataset foi dividido em dois subconjuntos: um conjunto de treinamento e um conjunto de validação. O conjunto de treinamento correspondeu a 80% das imagens, enquanto o conjunto de validação correspondeu aos 20% restantes. Essa divisão permitiu que o modelo aprendesse a partir de uma parte dos dados e depois fosse avaliado com imagens que não foram utilizadas diretamente durante o treinamento.

Para o treinamento, foi utilizado o otimizador adam, pois é um algoritmo amplamente utilizado em redes neurais e permite ajustar os pesos do modelo de forma eficiente durante o processo de aprendizagem. A função de perda utilizada foi `binary_crossentropy`, já que o problema corresponde a uma classificação binária, ou seja, o modelo deve escolher entre duas classes possíveis: real ou fake.

A principal métrica utilizada foi a acurácia, também conhecida como `accuracy`. Essa métrica indica a proporção de previsões corretas realizadas pelo modelo em relação ao total de imagens avaliadas.

O modelo foi treinado durante **10 épocas**. Uma época representa uma passagem completa do modelo pelo conjunto de dados de treinamento. Durante cada época, o modelo ajusta seus parâmetros internos com o objetivo de reduzir o erro e melhorar sua capacidade de classificação.

Ao longo do treinamento, foram registradas duas métricas principais: a acurácia e a perda. A acurácia permitiu observar o percentual de acertos do modelo, enquanto a perda permitiu analisar o nível de erro durante o processo de aprendizagem.

Os gráficos de acurácia e perda permitiram visualizar a evolução do modelo durante o treinamento. De modo geral, foi observada uma melhora progressiva no desempenho do

modelo, indicando que a rede neural conseguiu aprender padrões visuais relevantes a partir das imagens do dataset.

Ao final do treinamento, o modelo alcançou uma acurácia próxima de **89%** no conjunto de treinamento e aproximadamente **90%** no conjunto de validação. Esses resultados indicam um desempenho satisfatório para uma primeira aproximação acadêmica ao problema de classificação de logos reais e falsos.

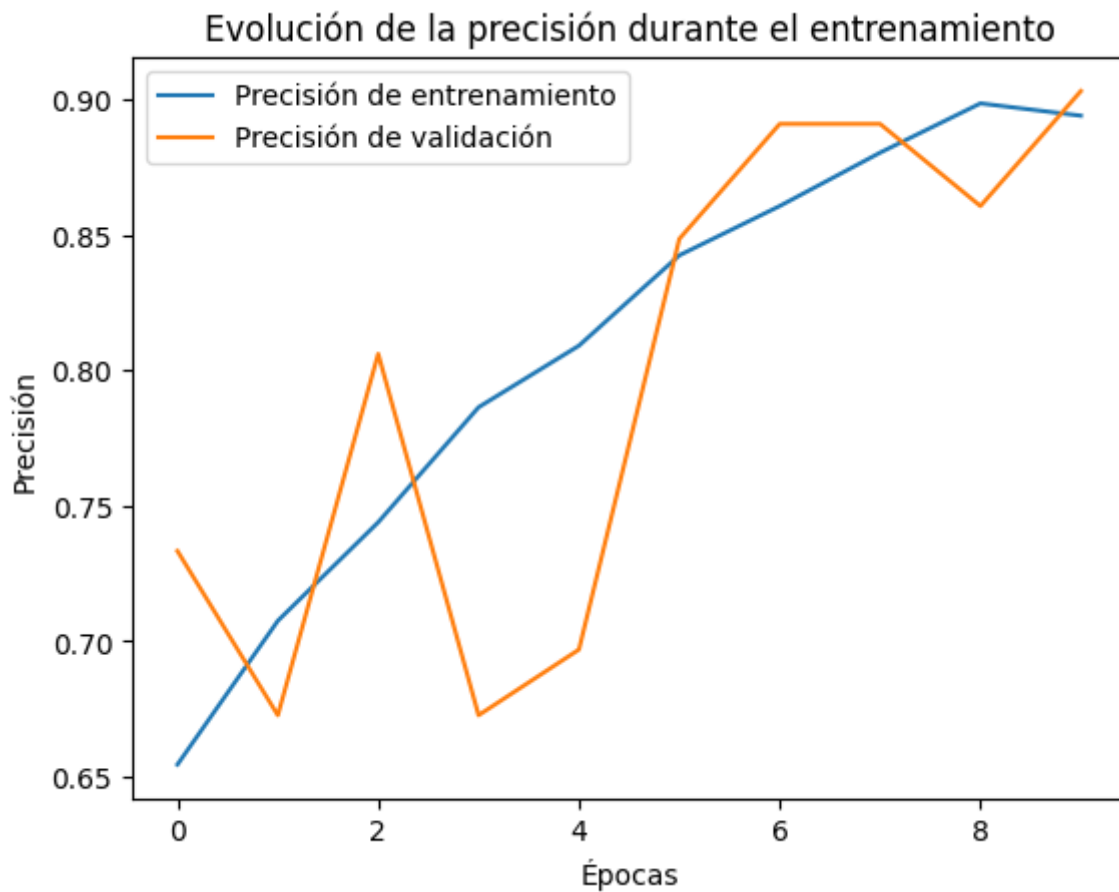


Imagem3. Evolução precisa Durante o Treinamento.

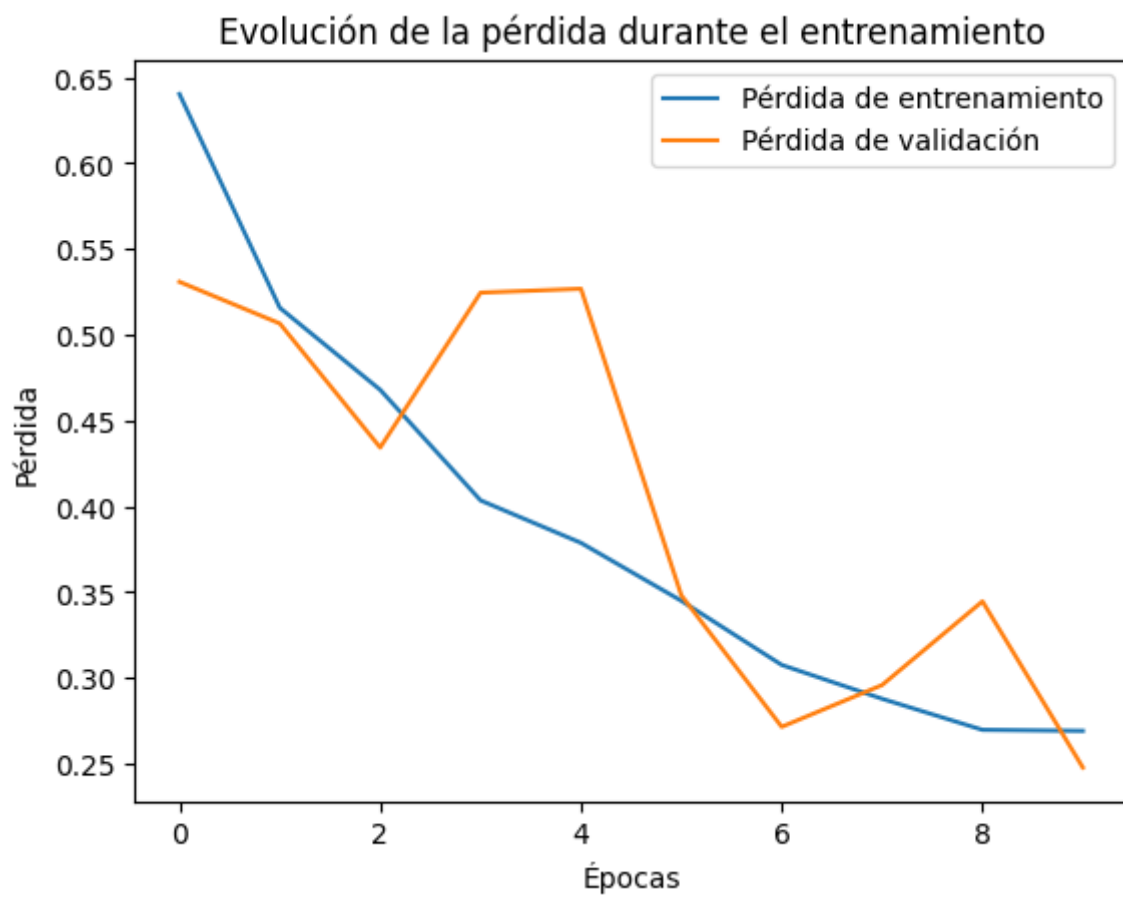


Imagem4. Evolução de perca durante o treinamento.

8. Resultados obtidos e métricas de avaliação

Após o treinamento, o modelo foi avaliado utilizando o conjunto de validação, que correspondeu a 20% do total de imagens. Essa avaliação permitiu medir o desempenho do modelo com dados que não foram utilizados diretamente durante o treinamento.

O modelo alcançou uma acurácia geral próxima de **90%** no conjunto de validação. Esse resultado indica que, de modo geral, o modelo conseguiu classificar corretamente a maioria das imagens avaliadas.

Além da acurácia geral, foi utilizada uma matriz de confusão para analisar com mais detalhes os acertos e erros do modelo. A matriz de confusão permite observar quantas imagens foram classificadas corretamente ou incorretamente em cada classe.

Os resultados obtidos foram os seguintes:

121 imagens fake foram classificadas corretamente como fake.

0 imagens fake foram classificadas incorretamente como real.

28 imagens reais foram classificadas corretamente como real.

16 imagens reais foram classificadas incorretamente como fake.

A partir desses resultados, observa-se que o modelo teve um desempenho muito alto na identificação de logos falsos, já que classificou corretamente todos os exemplos da classe fake presentes no conjunto de validação. No entanto, apresentou maior dificuldade com a classe real, pois algumas imagens reais foram classificadas incorretamente como falsas.

Também foram analisadas métricas como precision, recall e f1-score. Para a classe fake, o modelo obteve um recall de **1.00**, indicando que conseguiu identificar todos os logos falsos avaliados. Em contrapartida, para a classe real, o recall foi de aproximadamente **0.64**, mostrando que o modelo teve mais dificuldade para reconhecer corretamente todos os logos reais.

Essa diferença pode estar relacionada ao desbalanceamento do dataset, já que a classe fake continha 550 imagens, enquanto a classe real continha 275 imagens. Por haver mais exemplos de uma classe, o modelo pode aprender com maior facilidade os padrões associados a ela.

Em conclusão, os resultados obtidos foram satisfatórios para uma primeira aproximação acadêmica, já que o modelo alcançou uma acurácia geral alta. No entanto, a análise detalhada das métricas mostra que ainda existem limitações, especialmente na classificação de logos reais.

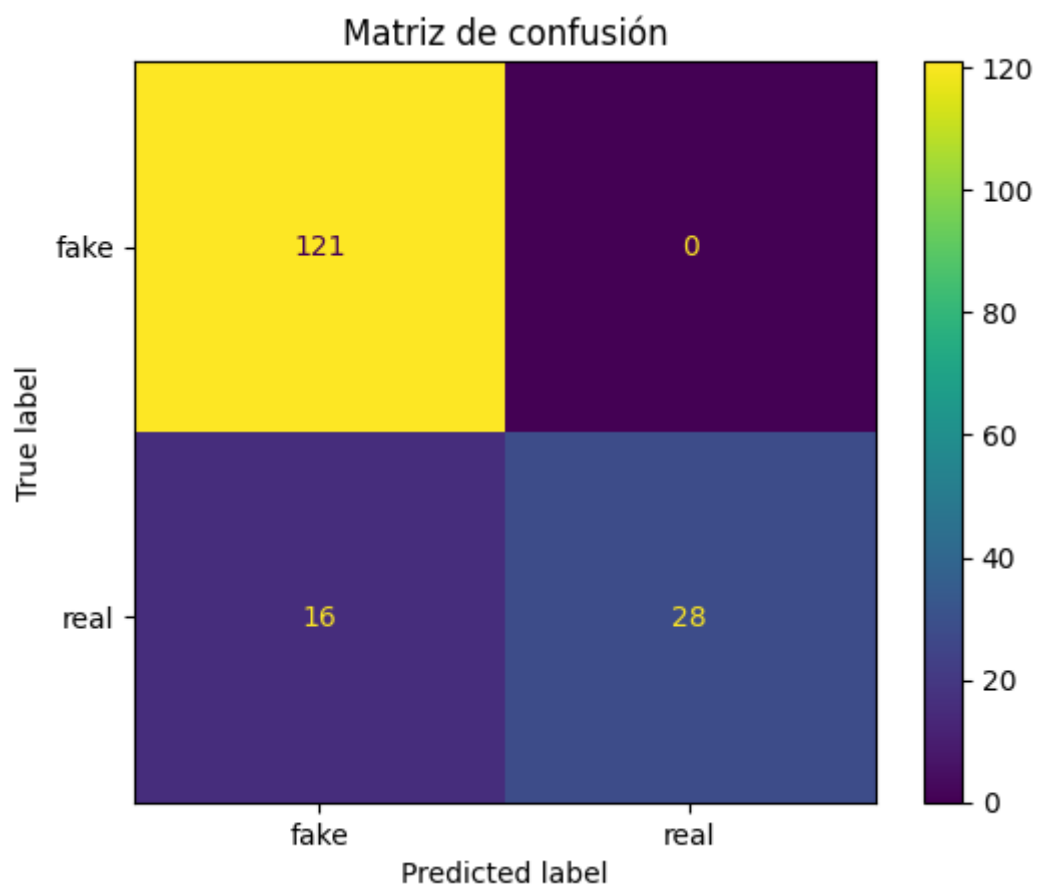


Imagem5. Matriz de confusão do modelo no conjunto de validação.

9. Testes com imagens externas

Além de avaliar o modelo com o conjunto de validação, também foram realizados testes com imagens externas ao dataset. O objetivo desses testes foi observar como o modelo se comporta diante de imagens que não foram utilizadas durante o treinamento.

Para realizar essa etapa, foi utilizada uma função no Google Colab que permite enviar uma imagem manualmente e encaminhá-la ao modelo para realizar uma predição. Antes de ser analisada, a imagem foi redimensionada para o mesmo tamanho utilizado durante o treinamento, ou seja, **70x70 pixels**.

Como o modelo foi treinado para uma tarefa de classificação binária, ele sempre retorna uma das duas possíveis respostas:

real
fake

Isso significa que, mesmo ao enviar uma imagem externa ou diferente das imagens do dataset, o modelo tentará classificá-la obrigatoriamente como real ou fake.

Durante os testes, observou-se que o modelo pode classificar corretamente algumas imagens de logos, mas também pode gerar predições incorretas quando a imagem está muito modificada ou não se parece com os exemplos utilizados durante o treinamento.

Um caso observado foi o uso de uma imagem modificada de um logo, com cores e textura diferentes dos logos originais. Embora visualmente, para uma pessoa, ela possa parecer uma versão alterada ou não oficial, o modelo pode classificá-la como real caso identifique padrões visuais semelhantes aos aprendidos durante o treinamento, como a forma geral do logo ou sua estrutura.

Isso demonstra uma limitação importante: o modelo não verifica se um logo é oficialmente autêntico do ponto de vista legal ou comercial. A predição é baseada apenas nos padrões visuais aprendidos a partir do dataset utilizado.

Portanto, os testes externos permitiram compreender que o modelo pode ser útil dentro do contexto do dataset e com imagens semelhantes às utilizadas durante o treinamento, mas não deve ser interpretado como uma ferramenta definitiva para validar a autenticidade de uma marca.

Essa observação é relevante para a análise crítica do projeto, pois mostra que o desempenho de um modelo de IA depende diretamente da qualidade, variedade e representatividade do conjunto de dados utilizado.

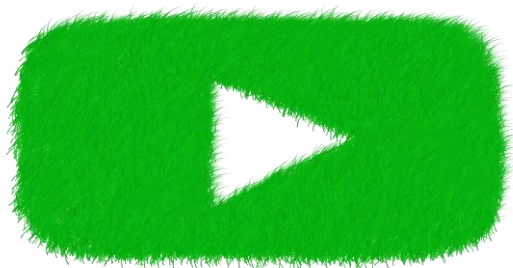


Imagem6. Logo Fake do Youtube.

Predicción de la IA: real

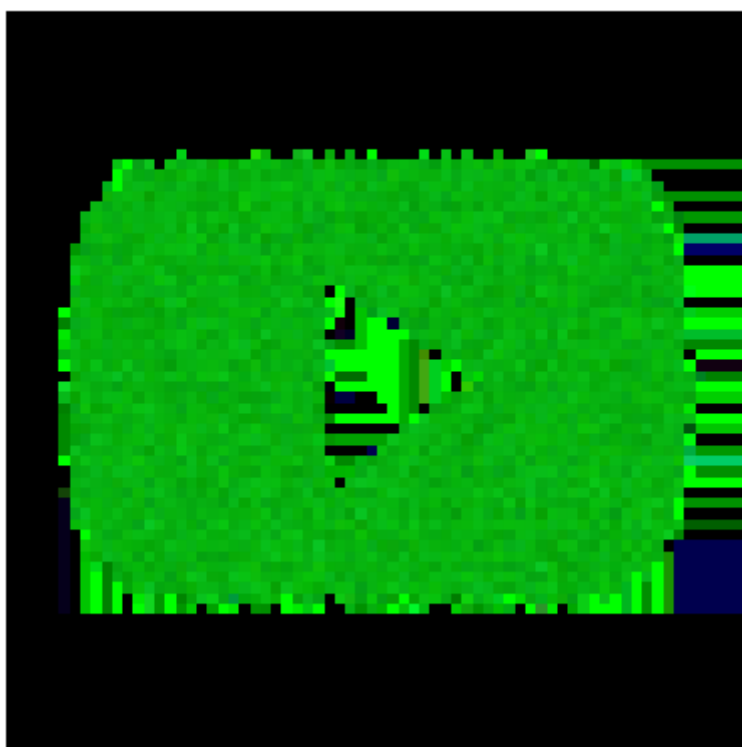


Imagem7. Imagem pós análise do Modelo.

10. Limitações do modelo

Embora o modelo tenha obtido um desempenho geral satisfatório durante a validação, é importante reconhecer que ele apresenta algumas limitações. Essas limitações devem ser consideradas ao interpretar os resultados e ao pensar em possíveis aplicações reais do modelo.

Uma das principais limitações identificadas foi o desbalanceamento do dataset. O conjunto de dados utilizado continha **550 imagens fake** e **275 imagens reais**, ou seja, havia o dobro de exemplos de logos falsos em comparação com os logos reais. Essa diferença pode ter influenciado o aprendizado do modelo, fazendo com que ele tivesse maior facilidade para reconhecer a classe fake do que a classe real.

Outra limitação importante foi a resolução das imagens. Durante o pré-processamento, todas as imagens foram redimensionadas para **70x70 pixels**. Esse tamanho permitiu reduzir o custo computacional e facilitar o treinamento no Google Colab, mas também provocou a perda de detalhes visuais importantes. Em alguns casos, pequenas características de um logo podem desaparecer ou se tornar difíceis de identificar após a redução de tamanho.

Também deve ser considerado que o modelo foi treinado apenas com duas classes: real e fake. Isso significa que o modelo sempre tentará classificar qualquer imagem dentro de uma dessas duas categorias, mesmo quando a imagem não corresponder a um logo. Por exemplo, se for enviada uma imagem de uma pessoa, uma paisagem ou um objeto, o modelo ainda entregará uma predição como real ou fake, mesmo que essa resposta não seja confiável.

Além disso, o modelo não funciona como um verificador oficial de autenticidade de marcas. Sua classificação se baseia nos padrões visuais aprendidos durante o treinamento, mas não é capaz de confirmar se um logo é legalmente autêntico, se pertence oficialmente a uma empresa ou se foi autorizado por uma marca.

Durante os testes com imagens externas, observou-se que o modelo pode cometer erros quando recebe imagens muito diferentes das presentes no dataset. Isso ocorre porque o desempenho de uma Inteligência Artificial depende diretamente dos exemplos utilizados durante seu treinamento. Caso uma imagem externa contenha cores, texturas ou modificações que o modelo não aprendeu previamente, a predição pode ser incorreta.

Por essas razões, o modelo deve ser entendido como uma primeira aproximação acadêmica ao problema de classificação de logos reais e falsos. Seus resultados são úteis para analisar o processo de treinamento e avaliação de uma IA, mas não devem ser utilizados como única ferramenta para tomar decisões reais sobre a autenticidade de marcas.

11. Aplicação no setor produtivo

O modelo desenvolvido pode ser relacionado a diferentes setores produtivos nos quais o uso de imagens, marcas e identidade visual possui um papel importante. Embora se trate de uma primeira aproximação acadêmica, a classificação de logos reais e falsos pode ter aplicações em áreas como design gráfico, marketing digital, comércio eletrônico, publicidade e proteção da identidade de marca.

No setor de design gráfico, um modelo desse tipo poderia ser utilizado como uma ferramenta de apoio para analisar logos e detectar possíveis alterações visuais. Isso pode ser útil em processos de revisão de identidade corporativa, nos quais é importante manter a coerência das cores, formas e elementos visuais de uma marca.

Na área de marketing digital e publicidade, a detecção de logos alterados poderia ajudar a identificar conteúdos visuais que não respeitam corretamente a imagem de uma empresa. Em redes sociais, anúncios digitais ou campanhas visuais, muitas marcas precisam garantir que seus logos sejam utilizados de forma adequada, evitando versões modificadas ou não autorizadas.

No comércio eletrônico, esse tipo de modelo poderia funcionar como uma ferramenta inicial para apoiar a detecção de possíveis produtos falsificados ou publicações suspeitas. Por exemplo, em plataformas de venda online, um sistema de análise de imagens poderia ajudar a identificar logos que visualmente se afastam dos padrões esperados de uma marca.

No entanto, é importante destacar que o modelo não deve ser utilizado como uma ferramenta definitiva para validar a autenticidade de uma marca. Sua função seria servir como apoio inicial dentro de um processo mais amplo de revisão, no qual também deveria participar uma pessoa especializada ou um sistema mais avançado de verificação.

Portanto, a principal aplicação do modelo no setor produtivo seria como uma ferramenta de análise preliminar. Essa ferramenta poderia ajudar a filtrar imagens suspeitas, reduzir o tempo de revisão manual e apoiar processos relacionados à identidade visual, controle de qualidade e proteção de marca.

Futuramente, com um dataset mais amplo, imagens de maior qualidade e um modelo mais robusto, esse tipo de solução poderia ser integrado a sistemas de monitoramento de marcas, plataformas de comércio eletrônico, ferramentas de design ou sistemas de revisão automática de conteúdo visual.

12. Melhorias futuras

A partir dos resultados obtidos e das limitações identificadas durante o desenvolvimento do projeto, é possível propor diferentes melhorias para futuras versões do modelo.

Uma primeira melhoria seria utilizar um dataset mais amplo e variado. O conjunto de dados utilizado neste projeto permitiu treinar o modelo de forma satisfatória, mas continha uma quantidade limitada de imagens. Um dataset maior poderia permitir que o modelo aprendesse uma variedade mais ampla de padrões visuais, aumentando sua capacidade de generalização diante de imagens novas.

Outra melhoria importante seria equilibrar a quantidade de imagens por classe. Neste projeto, a classe fake possuía 550 imagens, enquanto a classe real possuía 275 imagens. Esse desbalanceamento pode ter influenciado o comportamento do modelo, fazendo com que ele tivesse maior facilidade para reconhecer logos falsos. Para melhorar esse aspecto, seria possível aumentar a quantidade de imagens reais ou aplicar técnicas de balanceamento de dados.

Também seria recomendável utilizar imagens com maior resolução. Durante este projeto, as imagens foram redimensionadas para **70x70 pixels** para facilitar o processamento no Google Colab. No entanto, essa redução pode provocar perda de detalhes importantes. Em uma versão futura, poderiam ser utilizadas imagens de maior tamanho, como **128x128** ou **224x224 pixels**, para preservar mais informações visuais.

Outra possível melhoria seria aplicar técnicas de aumento de dados, conhecidas como data augmentation. Essas técnicas permitem gerar variações das imagens existentes por meio de rotações, mudanças de escala, ajustes de brilho, recortes ou pequenos deslocamentos. Isso poderia ajudar o modelo a aprender melhor e reduzir o risco de sobreajuste.

Além disso, poderia ser adicionada uma terceira classe para imagens que não correspondem a logos. Neste projeto, o modelo só podia classificar entre real e fake, por isso sempre entregava uma dessas duas respostas, mesmo quando a imagem enviada não era um logo. Uma classe adicional, como `no_logo` ou imagem inválida, permitiria que o modelo identificasse melhor imagens fora do domínio esperado.

Também seria possível testar o uso de modelos mais avançados por meio de transfer learning. Essa técnica consiste em utilizar modelos previamente treinados com grandes conjuntos de imagens, como MobileNet, ResNet ou EfficientNet, e adaptá-los ao problema específico de classificação de logos reais e falsos. Isso poderia melhorar o desempenho do modelo, especialmente ao trabalhar com imagens de maior resolução.

Por fim, seria recomendável realizar testes com um conjunto externo de imagens que não pertença ao dataset original. Isso permitiria avaliar de maneira mais realista o desempenho do modelo diante de imagens novas e verificar se ele realmente consegue generalizar além dos dados utilizados durante o treinamento.

Em resumo, as principais melhorias futuras seriam ampliar e equilibrar o dataset, utilizar imagens com maior resolução, aplicar aumento de dados, incorporar uma classe para imagens fora do domínio, testar modelos pré-treinados e avaliar o sistema com imagens externas mais variadas.

Conclusão

O presente projeto permitiu desenvolver uma primeira aproximação ao treinamento de um modelo de Inteligência Artificial aplicado à classificação de imagens de logos reais e falsos. Por meio do uso do Google Colab, Python, TensorFlow/Keras e de um dataset obtido no Kaggle, foi possível construir, treinar e avaliar uma rede neural convolucional para resolver um problema de classificação binária.

Durante o desenvolvimento do trabalho, foram realizadas a descarga do dataset, a análise de sua estrutura, a reorganização das imagens em duas classes, o pré-processamento dos dados, a criação do modelo, o treinamento e a avaliação por meio de métricas de desempenho. Esse processo permitiu compreender, de forma prática, as principais etapas envolvidas no desenvolvimento de um modelo de IA aplicado a imagens.

Os resultados obtidos foram satisfatórios para uma primeira aproximação acadêmica. O modelo alcançou uma acurácia próxima de 90% no conjunto de validação, o que indica que conseguiu classificar corretamente a maioria das imagens avaliadas. Além disso, a matriz de confusão mostrou que o modelo teve um desempenho especialmente alto na identificação de logos falsos.

No entanto, também foram identificadas limitações importantes. O dataset utilizado estava desbalanceado, pois continha mais imagens falsas do que reais. Além disso, as imagens foram trabalhadas em uma resolução reduzida de 70x70 pixels, o que pode ter provocado perda de detalhes visuais. Também foi observado que o modelo não funciona como um verificador oficial de autenticidade de marcas, mas sim como um classificador baseado em padrões aprendidos a partir do dataset.

A partir dessas observações, conclui-se que o modelo pode ser útil como uma ferramenta inicial de apoio para analisar imagens de logos, mas não deve ser utilizado como única fonte de decisão em contextos reais. Para melhorar seu desempenho, seria necessário ampliar o dataset, equilibrar melhor as classes, utilizar imagens de maior resolução, testar modelos mais avançados e adicionar uma categoria para imagens que não correspondam a logos.

Em conclusão, o projeto permitiu cumprir o objetivo de treinar e avaliar um modelo de IA no Google Colab, além de possibilitar uma reflexão sobre seus resultados,

limitações e possíveis aplicações em setores relacionados ao design gráfico, marketing digital, comércio eletrônico e proteção da identidade visual de marcas.

Referencias

Google. (s. f.). *Google Colaboratory*. <https://colab.research.google.com/>

Kaggle. (s. f.). *Kaggle API documentation*. <https://www.kaggle.com/docs/api>

Prosperchuks. (s. f.). *Fake/Real Logo Detection Dataset*. Kaggle.
<https://www.kaggle.com/datasets/prosperchuks/fakereal-logo-detection-dataset>

TensorFlow. (s. f.). *Keras: The high-level API for TensorFlow*.
<https://www.tensorflow.org/guide/keras>

TensorFlow. (s. f.). *Image classification*.
<https://www.tensorflow.org/tutorials/images/classification>

Scikit-learn. (s. f.). *Classification metrics*. https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics